AI Assisted Coding

Assignment-12.3

Name: K. Sai Krishna Reddy

Rollno: 2303A52305

Batch: 45

Task 1: Sorting Student Records for Placement Drive

Scenario

SR University's Training and Placement Cell needs to shortlist candidates efficiently during campus placements. Student records must be sorted by CGPA in descending order.

Tasks

1. Use GitHub Copilot to generate a program that stores student records (Name, Roll Number, CGPA).

2. Implement the following sorting algorithms using AI assistance:

o Quick Sort

o Merge Sort

3. Measure and compare runtime performance for large datasets.

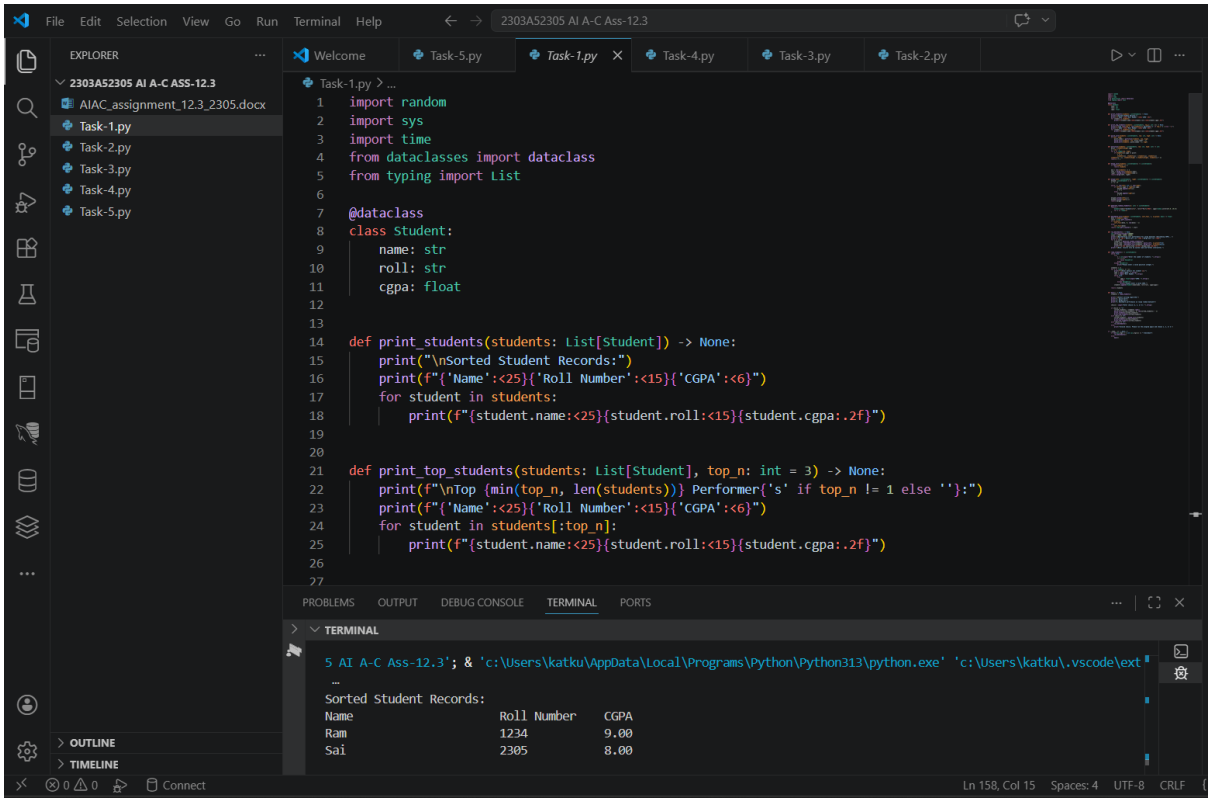
4. Write a function to display the top 10 students based on CGPA.

Expected Outcome

• Correctly sorted student records.

• Performance comparison between Quick Sort and Merge Sort.

• Clear output of top-performing students.



Task 2: Implementing Bubble Sort with AI Comments

• Task: Write a Python implementation of Bubble Sort.

• Instructions:

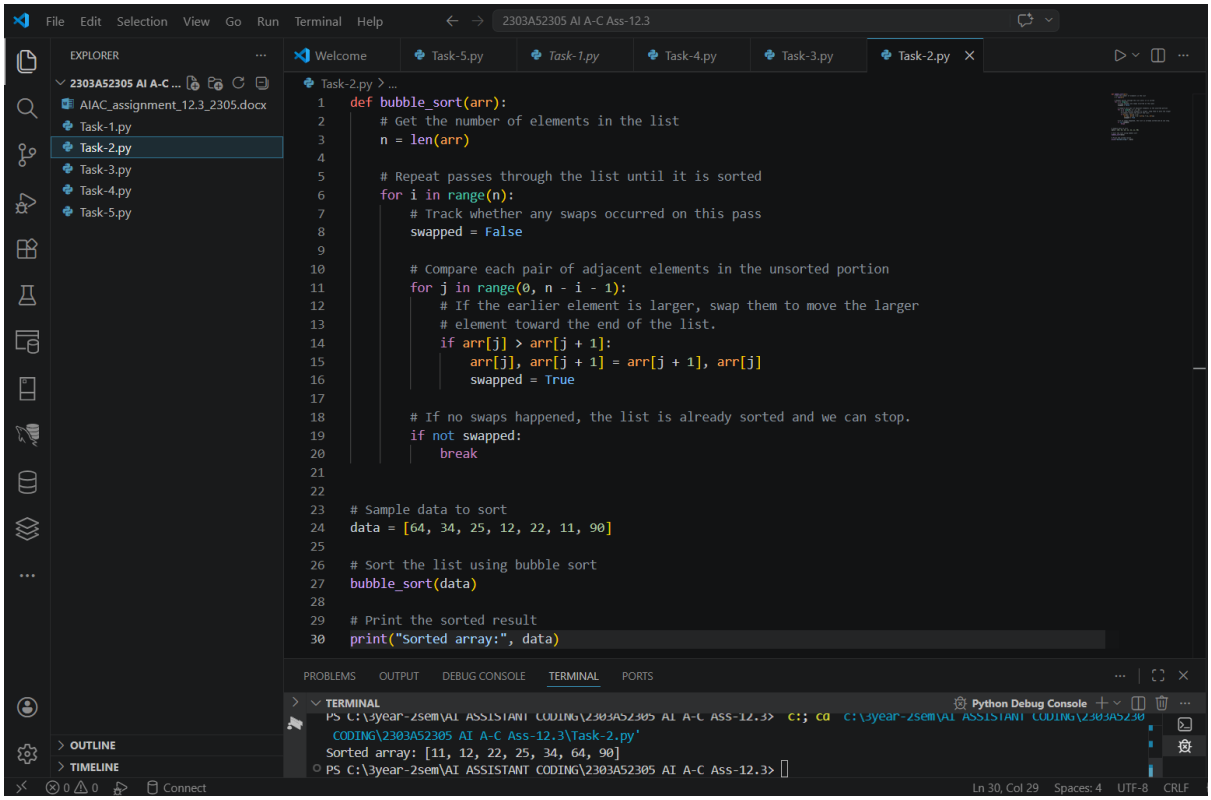
• Students implement Bubble Sort normally.

• Ask AI to generate inline comments explaining key logic (like swapping, passes, and termination).

• Request AI to provide time complexity analysis.

• Expected Output:

• A Bubble Sort implementation with AI-generated explanatory comments and complexity analysis.



Task 3: Quick Sort and Merge Sort Comparison

• Task: Implement Quick Sort and Merge Sort using recursion.

• Instructions:

• Provide AI with partially completed functions for recursion.

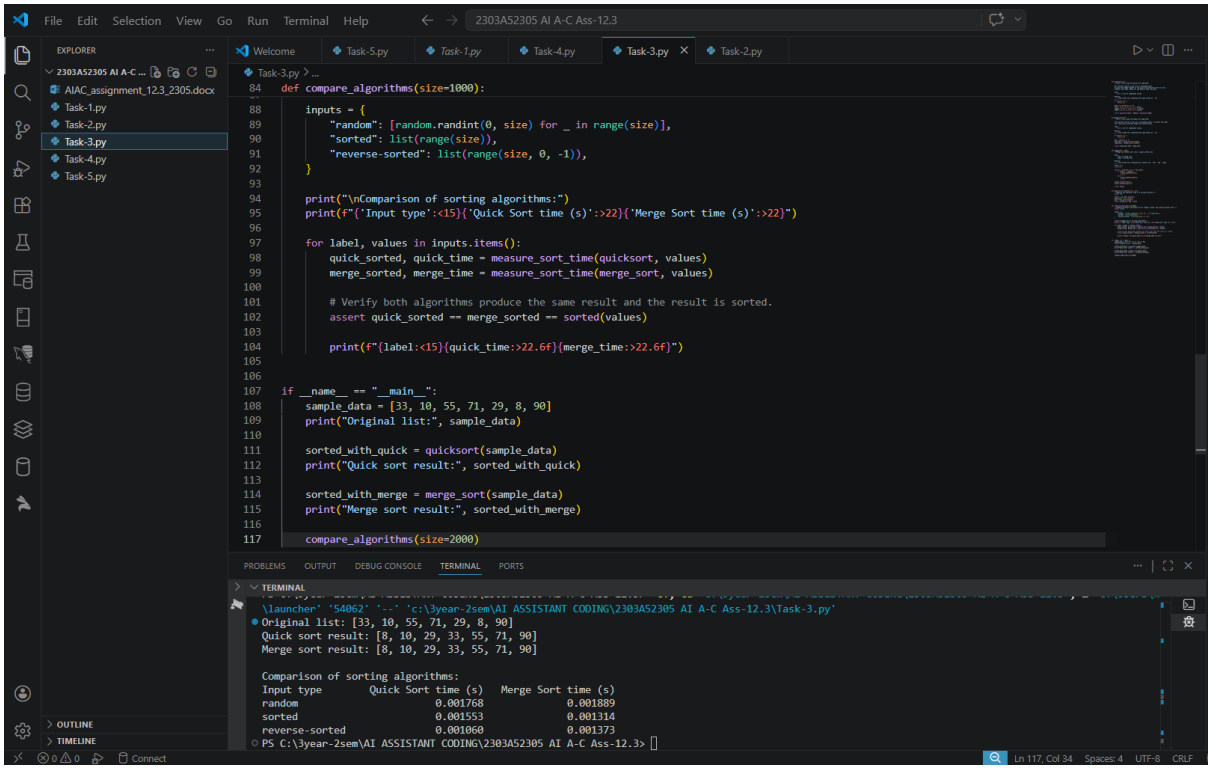
• Ask AI to complete the missing logic and add docstrings.

• Compare both algorithms on random, sorted, and reverse-sorted lists.

• Expected Output:

• Working Quick Sort and Merge Sort implementations.

• AI-generated explanation of average, best, and worst-case complexities.



Task 4 (Real-Time Application – Inventory Management System)

Scenario: A retail store's inventory system contains thousands of products, each with attributes like product ID, name, price, and stock quantity. Store staff need to:

1. Quickly search for a product by ID or name.

2. Sort products by price or quantity for stock analysis.

Task:

• Use AI to suggest the most efficient search and sort algorithms for this use case.

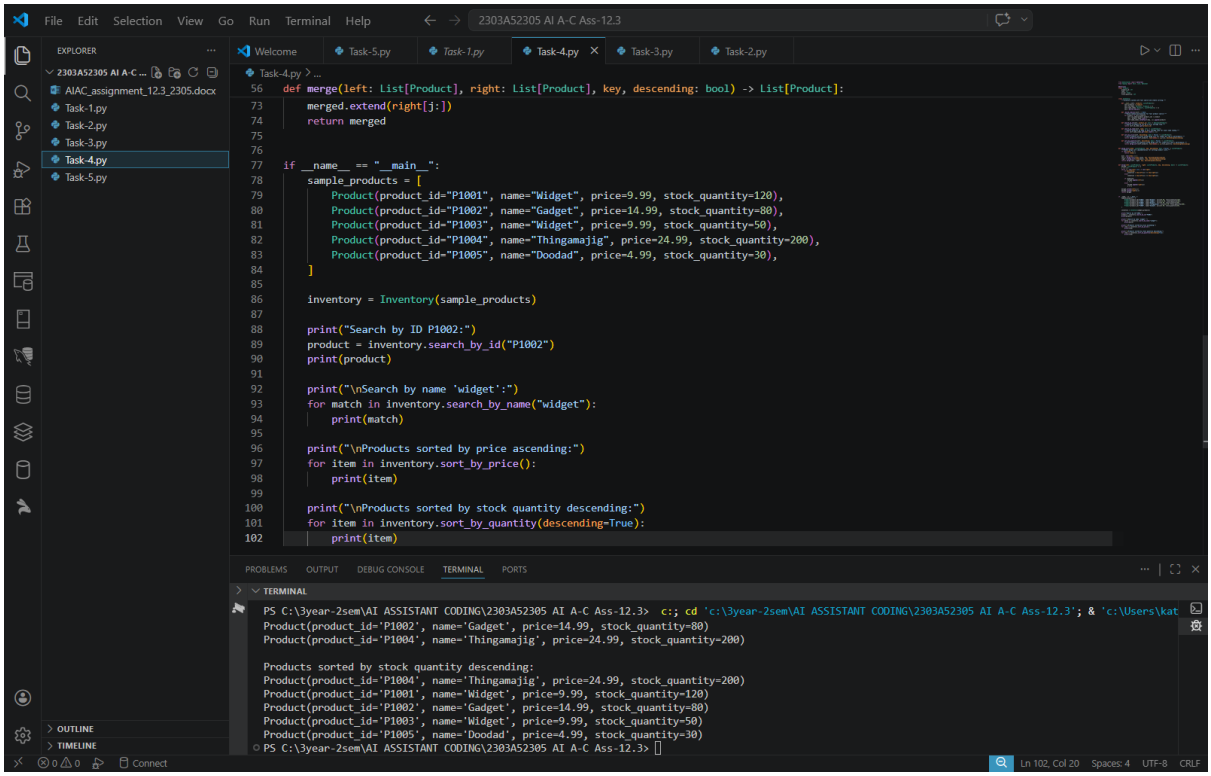
• Implement the recommended algorithms in Python.

• Justify the choice based on dataset size, update frequency, and performance requirements.

Expected Output:

• A table mapping operation → recommended algorithm → justification.

• Working Python functions for searching and sorting the inventory.



Task 5: Real-Time Stock Data Sorting & Searching

Scenario:

An AI-powered FinTech Lab at SR University is building a tool for analyzing stock price movements. The requirement is to quickly sort stocks by daily gain/loss and search for specific stock symbols efficiently.

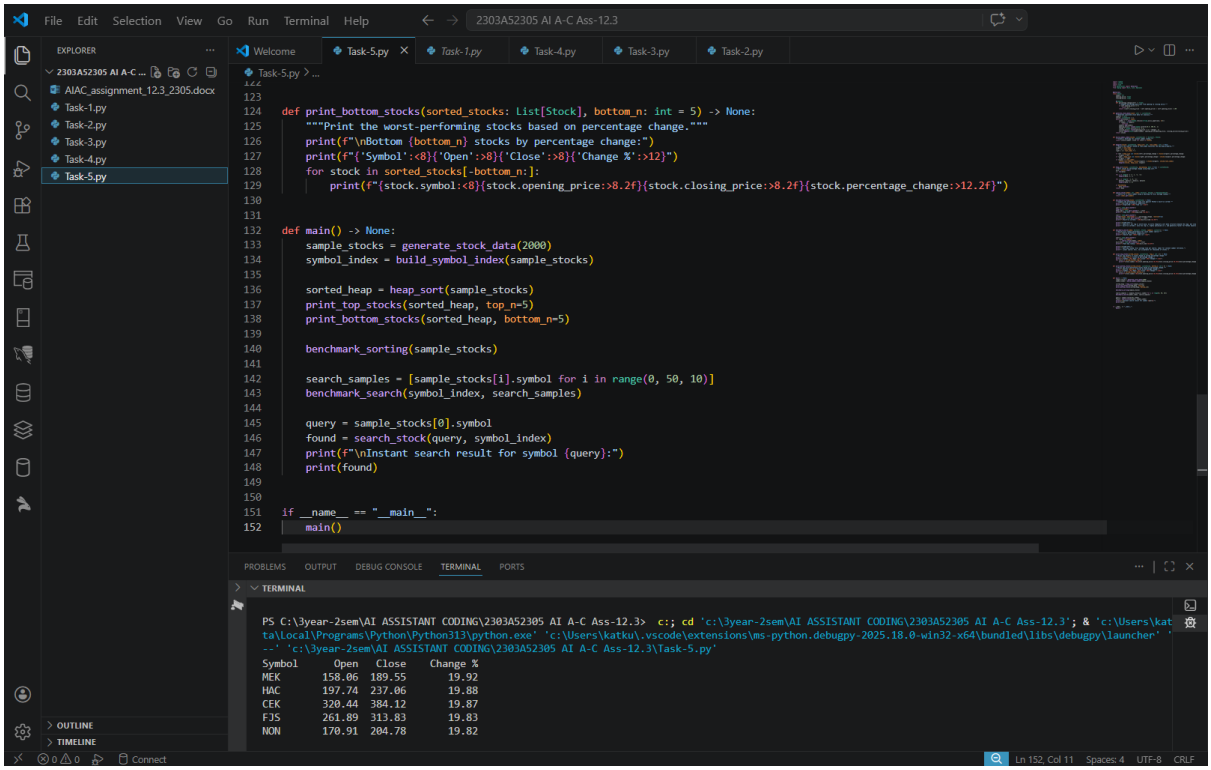
• Use GitHub Copilot to fetch or simulate stock price data (Stock Symbol, Opening Price, Closing Price).

• Implement sorting algorithms to rank stocks by percentage change.

• Implement a search function that retrieves stock data instantly when a stock symbol is entered.

• Optimize sorting with Heap Sort and searching with Hash Maps.

• Compare performance with standard library functions (sorted(), dict lookups) and analyze trade-offs.


File Edit Selection View Go Run Terminal Help
2303A52305 AI A-C Ass-12.3
EXPLORER
2303A52305 AI A-C ...
AIAC_assignment_12.3_2305.docx
Task-1.py
Task-2.py
Task-3.py
Task-4.py
Task-5.py
Welcome
Task-5.py
Task-1.py
Task-4.py
Task-3.py
Task-2.py
Task-5.py > ...
def print_bottom_stocks(sorted_stocks: List[Stock], bottom_n: int = 5) -> None:
"""Print the worst-performing stocks based on percentage change."""
print(f"\nBottom {bottom_n} stocks by percentage change:")
print(f"{'Symbol':<8}{'Open':>8}{'Close':>8}{'Change %':>12}")
for stock in sorted_stocks[-bottom_n:]:
print(f"{stock.symbol:<8}{stock.opening_price:>8.2f}{stock.closing_price:>8.2f}{stock.percentage_change:>12.2f}")
def main() -> None:
sample_stocks = generate_stock_data(2000)
symbol_index = build_symbol_index(sample_stocks)
sorted_heap = heap_sort(sample_stocks)
print_top_stocks(sorted_heap, top_n=5)
print_bottom_stocks(sorted_heap, bottom_n=5)
benchmark_sorting(sample_stocks)
search_samples = [sample_stocks[i].symbol for i in range(0, 50, 10)]
benchmark_search(symbol_index, search_samples)
query = sample_stocks[0].symbol
found = search_stock(query, symbol_index)
print(f"\nInstant search result for symbol {query}:")
print(found)
if __name__ == "__main__":
main()
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
TERMINAL
PS C:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3> c:; cd 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3'; & 'c:\Users\katku\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\katku\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '--' 'c:\3year-2sem\AI ASSISTANT CODING\2303A52305 AI A-C Ass-12.3\Task-5.py'
Symbol Open Close Change %
MEK 158.06 189.55 19.92
HAC 197.74 237.06 19.88
CEK 320.44 384.12 19.87
FJS 261.89 313.83 19.83
NON 170.91 204.78 19.82
OUTLINE
TIMELINE
Connect
Ln 152, Col 11 Spaces: 4 UTF-8 CRLF